

10.1 Steps to set up Spring Boot + REST + JPA (Maven deps: spring-boot-starter-web, spring-boot-starter-data-jpa, MySQL connector) and role of embedded Tomcat.

Solution:

Spring Boot + REST + JPA Setup

1. Create Spring Boot Project

- Create a Maven Spring Boot project.
- Select Java and the required Spring Boot version.
- Configure Group, Artifact, and Java version.

2. Add Maven Dependencies

Add these dependencies in pom.xml:

```
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
  </dependency>
  <dependency>
    <groupId>com.mysql</groupId>
    <artifactId>mysql-connector-j</artifactId>
    <scope>runtime</scope>
  </dependency>
</dependencies>
```

3. Configure MySQL

In application.properties:

```
spring.datasource.url=jdbc:mysql://localhost:3306/student_db
spring.datasource.username=root
```

spring.datasource.password=1234

spring.jpa.hibernate.ddl-auto=update

4. Create JPA Entity

Create a class such as **Student** and use `@Entity`.

- Entity → Represents database table
- JPA → Maps Java objects to database records

5. Create Repository

Create an interface extending `JpaRepository`.

- It provides common operations such as save, find, update, and delete without writing SQL manually.

6. Create REST Controller

Use `@RestController` to create REST API endpoints.

Example:

`@RestController`

```
class StudentController {
```

```
    @GetMapping("/students")
```

```
    public String students() {
```

```
        return "Student List";
```

```
    }
```

```
}
```

7. Embedded Tomcat

- `spring-boot-starter-web` includes embedded Tomcat by default.
- It allows the Spring Boot application to run directly without installing/configuring an external Tomcat server.

10.2 JPA Student entity (id, name, cgpa) + StudentRepository extends JpaRepository.

Solution:

1.Student Entity:

```
import jakarta.persistence.*;
```

```

@Entity
public class Student {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int id;
    private String name;
    private double cgpa;
    public Student() {}
}

```

- `@Entity` → Maps the class to a database table.
- `@Id` → Defines id as the primary key.
- `@GeneratedValue` → Automatically generates the ID.

2. StudentRepository

```

import org.springframework.data.jpa.repository.JpaRepository;

public interface StudentRepository
    extends JpaRepository<Student, Integer> {
}

```

- `JpaRepository` provides built-in methods such as `save()`, `findAll()`, `findById()`, and `deleteById()`.
- Therefore, we do not need to write basic SQL queries manually.

Structure

Student Entity → StudentRepository → MySQL Database

10.3 @RestController with GET /students and POST /students endpoints using the repository.

Solution :

```

import org.springframework.web.bind.annotation.*;
import java.util.List;

@RestController
@RequestMapping("/students")
public class StudentController {
    private final StudentRepository repo;
}

```

```
public StudentController(StudentRepository repo) {  
    this.repo = repo;  
}  
// GET /students  
@GetMapping  
public List<Student> getStudents() {  
    return repo.findAll();  
}  
@PostMapping  
public Student addStudent(@RequestBody Student student) {  
    return repo.save(student);  
}  
}
```

Key Points:

- **@RestController** → Makes the class a REST API controller.
- **@GetMapping** → Handles GET /students and returns all students.
- **@PostMapping** → Handles POST /students and adds a new student.
- **@RequestBody** → Converts JSON request data into a Student object.
- **repo.findAll()** → Retrieves all students

11.1 Design DB schema for quiz on Crops / Geography / Academic Institutions of your district + Player/Score table. Justify design.

Solution:

Quiz Database Schema

For a quiz on Crops, Geography, and Academic Institutions of a district, we can design the database using separate tables for questions, players, and scores.

1. Questions Table

Field	Type	Description
question_id	INT, PK	Unique question ID
question_text	VARCHAR	Question
option_a	VARCHAR	Option A
option_b	VARCHAR	Option B
option_c	VARCHAR	Option C
option_d	VARCHAR	Option D
correct_option	CHAR	Correct answer

2. Players Table

Field	Type	Description
player_id	INT, PK	Unique player ID
player_name	VARCHAR	Player name

3. Scores Table

Field	Type	Description
score_id	INT, PK	Unique score ID
player_id	INT, FK	References Players
score	INT	Obtained score

Field	Type	Description
-------	------	-------------

quiz_date	DATE	Date of quiz
-----------	------	--------------

SQL Schema

```
CREATE DATABASE district_quiz;
```

```
USE district_quiz;
```

```
CREATE TABLE Questions (
```

```
    question_id INT PRIMARY KEY AUTO_INCREMENT,
```

```
    question_text VARCHAR(255),
```

```
    option_a VARCHAR(100),
```

```
    option_b VARCHAR(100),
```

```
    option_c VARCHAR(100),
```

```
    option_d VARCHAR(100),
```

```
    correct_option CHAR(1),
```

```
    category VARCHAR(50)
```

```
);
```

```
CREATE TABLE Players (
```

```
    player_id INT PRIMARY KEY AUTO_INCREMENT,
```

```
    player_name VARCHAR(100)
```

```
);
```

```
CREATE TABLE Scores (
```

```
    score_id INT PRIMARY KEY AUTO_INCREMENT,
```

```
    player_id INT,
```

```
    score INT,
```

```
    quiz_date DATE,
```

```
    FOREIGN KEY (player_id) REFERENCES Players(player_id)
```

```
);
```

Justification

- **Separate category field:** Allows questions to be grouped into Crops, Geography, and Academic Institutions.
- **Players and Scores are separate:** One player can have multiple quiz scores.
- **Primary Keys:** Ensure every question, player, and score has a unique identity.
- **Foreign Key:** Scores.player_id connects each score to the correct player.
- The design reduces data duplication and makes searching, updating, and generating leaderboards easier.

11.2 Servlet method: save Player name + final score into PlayerScore via JDBC/PreparedStatement.

Solution: Servlet Method to Save Player Score :The Servlet receives the Player Name and Final Score and stores them in the PlayerScore table using JDBC and PreparedStatement.

```
protected void doPost(HttpServletRequest request,
                        HttpServletResponse response)
    throws ServletException, IOException {

    String name = request.getParameter("name");
    int score = Integer.parseInt(request.getParameter("score"));
    String url = "jdbc:mysql://localhost:3306/district_quiz";
    String user = "root";
    String password = "1234";
    String sql = "INSERT INTO PlayerScore(player_name, score) VALUES (?, ?)";
    try {
        Connection con = DriverManager.getConnection(url, user, password);
        PreparedStatement ps = con.prepareStatement(sql);
        ps.setString(1, name);
        ps.setInt(2, score);
        ps.executeUpdate();
        response.getWriter().println("Score saved successfully.");
    }
```

```

        ps.close();

        con.close();

    } catch (SQLException e) {

        response.getWriter().println("Database Error: " + e.getMessage());

    }

}

```

Key Points

- **getParameter()** reads the player's name and final score.
- **PreparedStatement** safely inserts the data.
- **executeUpdate()** saves the record in PlayerScore.
- **try-catch** handles database errors.

11.3 Quiz logic: ≥ 3 MCQs (crop, geography, institution) as objects; present, check answer, increment score.

Solution :

Quiz Logic Using Java Objects

```

class Question {

    String question, a, b, c, d, correct;

    Question(String q, String a, String b, String c,

        String d, String correct) {

        this.question = q;

        this.a = a; this.b = b; this.c = c;

        this.d = d; this.correct = correct;

    }

}

public class Quiz {

    public static void main(String[] args) {

```



```

Question[] questions = {
    new Question(
        "Which is a major crop of Bangladesh?",
        "Rice", "Tea", "Coffee", "Cocoa", "A"),
    new Question(
        "Which district is famous for Mahasthangarh?",
        "Bogura", "Dhaka", "Khulna", "Sylhet", "A"),
    new Question(
        "Which is an academic institution?",
        "MBSTU", "Padma River", "Rice", "Hilsa", "A")
};

int score = 0;

for (Question q : questions) {
    System.out.println(q.question);
    System.out.println("A. " + q.a);
    System.out.println("B. " + q.b);
    System.out.println("C. " + q.c);
    System.out.println("D. " + q.d);
    String answer = "A";
    if (answer.equalsIgnoreCase(q.correct))
        score++;
}

System.out.println("Final Score: " + score);
}
}

```

12.1 List all 5 GoF Creational patterns (Singleton, Factory Method, Abstract Factory, Builder, Prototype) with one-line purpose each.

Solution : 5 GoF Creational Design Patterns

Creational patterns deal with how objects are created.

1. **Singleton** — Ensures that a class has only one object/instance and provides a global access point to it.
2. **Factory Method** — Defines a method for creating objects while allowing subclasses to decide which object to create.
3. **Abstract Factory** — Provides an interface for creating families of related objects without specifying their concrete classes.
4. **Builder** — Builds a complex object step-by-step, allowing different representations of the same object.
5. **Prototype** — Creates new objects by copying/cloning an existing object.

12.2 List all 7 GoF Structural patterns (Adapter, Bridge, Composite, Decorator, Facade, Flyweight, Proxy) with one-line purpose each.

Solution : 7 GoF Structural Design Patterns

Structural patterns deal with how classes and objects are combined to form larger structures.

1. **Adapter** — Converts the interface of one class into another interface that the client expects.
2. **Bridge** — Separates an abstraction from its implementation so both can vary independently.
3. **Composite** — Treats individual objects and groups of objects uniformly using a tree-like structure.
4. **Decorator** — Adds new responsibilities or features to an object dynamically without changing its class.
5. **Facade** — Provides a simple interface to a complex subsystem.
6. **Flyweight** — Reduces memory usage by sharing common objects/data among many objects.
7. **Proxy** — Provides a substitute or placeholder for another object to control access to it.

12.3 Implement Singleton (thread-safe) and Adapter with short, complete Java code examples.

Solution :

1. Singleton Pattern — Thread-Safe

Purpose: Ensures that a class has only one instance throughout the application.

```
class Singleton {  
    private static volatile Singleton instance;  
    private Singleton() {}  
    public static Singleton getInstance() {  
        if (instance == null) {  
            synchronized (Singleton.class) {  
                if (instance == null)  
                    instance = new Singleton();  
            }  
        }  
        return instance;  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Singleton s1 = Singleton.getInstance();  
        Singleton s2 = Singleton.getInstance();  
        System.out.println(s1 == s2); // true  
    }  
}
```

Why Thread-Safe?

- **volatile** ensures visibility between threads.

- **synchronized** prevents multiple threads from creating multiple objects.
- **Double-checked locking** improves performance by avoiding synchronization after the instance is created.

2. Adapter Pattern

Purpose: Allows **incompatible interfaces** to work together.

// Target interface

```
interface Printer {  
    void print(String text);  
}
```

// Existing incompatible class

```
class OldPrinter {  
    void oldPrint(String text) {  
        System.out.println(text);  
    }  
}
```

// Adapter

```
class PrinterAdapter implements Printer {  
    private OldPrinter printer = new OldPrinter();  
  
    public void print(String text) {  
        printer.oldPrint(text);  
    }  
}
```

// Client

```
public class Main {  
    public static void main(String[] args) {
```

```
Printer p = new PrinterAdapter();  
p.print("Hello World");  
}  
}
```

Adapter Working

Client → Printer → PrinterAdapter → OldPrinter

- **Printer** → Expected interface.
- **OldPrinter** → Existing incompatible class.
- **PrinterAdapter** → Converts the old interface to the required interface.